Practical 1

1) Title : Installation of Android Studio on Windows and Creating a hello world Application

2) Aim : To install Android Studio on Windows and create and run basic Android Application - "Hello World"

3) Procedure :

a. System Requirements

Ensure the system meets the following :

- Windows 10/11 (64 bit)
- Minimum 8GB RAM
- Atleast 10 - 15 GB of free disk space
- Virtualization enabled in BIOS (for emulator)

b. Download Android Studio

- Open web Browser
- Visit official website
- Click download Android Studio
- Accept terms and download

c. Install Android Studio :

- Run the .exe file
- Select components
  - Android Studio
  - Android Virtual Devices
- Choose installation location
- Click install
- Finish to launch Android Studio

d. Initial Setup

- Android Studio setup wizard will open
- Select Standard installation
- Choose UI theme (Light / Dark)
- Allow it to download : Android SDK, Emulator components
- Click finish after setup completes.

e. Create Hello World Application

- Open Android Studio
- Create new project
- Select Empty activity and click next
- Enter following Details :

Name	:	Hello World
Package Name	:	com.example.helloworld
language	:	Java / Kotlin
Minimum SDK	:	API 21 or higher

- Click Finish
- Wait for gradle build to complete

f. Run the Application

- Enable Developer Option on Phone
- Enable USB debugging
- Connect device via USB

g. Execute the App

- Click the Run button
- Selected connected device
- Wait for installation and launch

5) Results

Android Studio was successfully installed on Windows. A Hello World Application was created, compiled and executed successfully on a device displaying the "helloworld" message on screen.

RG

Practical - 2

1) Title: Security testing of DIVA Application using ADB for identifying Common Android Vulnerabilities

2) Aim: To analyse the DIVA android app using ADB and identify security vulnerabilities such as insecure logging, hardcoded information, etc

3) Procedure:

a. Setup Environment:

- Install Android Emulator
- Enable Developer Options → USB Debugging
- Verify adb connection:

```
adb devices.
```

- Install DIVA apk

```
adb install diva-beta.apk
```

b. Launch the application and start Vulnerability testing

i) Insecure logging

- Open log Monitoring:

```
adb logcat
```

- Perform actions inside DIVA (login, data entry)
- filter logs:

```
adb logcat | findstr diva
```

- Observe for the following

- Usernames / passwords in logs
- Sensitive data printed
- Debug messages exposing internal logic

(ii) Hardcoded information detection

. Pull the APK

adb shell pm path jakhar.aseem.diva
adb pull <apk-path>

. Extract readable strings such as password, key, token

strings base.apk | grep -i password
strings base.apk | grep -i key
strings base.apk | grep -i token

. Search for hardcoded values:

- API Keys
- Credentials
- Secret strings

(iii) Insecure Data Storage

- Open ADB shell

adb shell

- Run as user

run-as jakhar.aseem.diva

- View files

ls

- Check

shared_prefs
databases
files

- Pull data using adb pull command
- Open files and check for plaintext credential / sensitive data

(iv) Input validation

- Open Input Validation Challenge
- Enter unexpected inputs such as:
  - SQL injection payload ' OR 1=1 --
- Observe application behaviour

(v) Access Control

- List app activities

  adb shell dumpsys package jakhar.aseem.diva | findstr Activity

- Attempt to launch restricted activities directly

  adb shell am start -n jakhar.aseem.diva/.<Activity name>

- As observed restricted screen opens without authentication

5) ~~Data~~ Result:

- Sensitive information was observed in logcat indicating insecure logging
- Hardcoded credentials and secrets were identified within the application
- Application data was ~~accessible~~ accessible from device storage in plaintext, showing insecure data storage
- Improper input validation allowed malicious / unexpected inputs
- Certain internal components were accessible directly, indicating weak access control

Practical -3

1) Title:
Perform static analysis of mobile application using MobSF

2) Aim:
To perform static analysis of a mobile app using Mobile framework and identify vulnerabilities, permission and security issues in the APK/IPA file

3) Procedure
i) Installation of MobSF
ii) Upload the application
Open Mobsf web interface in browser
Upload the target APK or IPA file
iii) Perform static Analysis
- MobSF automatically extracts, decompile the app.
- It analyzes manifest files, permissions, API calls, hardcoded secret, and code
iv) Review Security Findings
- Check for dangerous permission
- Identify insecure API usage, hardcoded credential or weak cryptography
- Review code quality and potential vulnerabilities
v) Generate Report
- Download the detailed analysis report generated by MobSF
- Document the findings for further communication

4) Result:

Successfully performed static analysis of the given mobile application MOBSF

Practical - 4

1) Title: Perform Dynamic analysis of Mobile Application

2) Aim:
To perform dynamic analysis of mobile application using MobSF for monitoring runtime behaviour, network traffic, API calls

3) Procedure:

i) Setup MobSF Dynamic Analysis Emulator

- Install MobSF, configure the dynamic analysis environment
- Ensure required dependencies such as VirtualBox and android SDK are installed.

ii) Start MobSF Dynamic Analyzer

- Launch MobSF, select the dynamic analysis.
- Connect the emulator / device to MobSF

iii) Deploy the app.

- Install the target APK on the emulator / device through MobSF
- Start the Application

iv) Monitor Runtime Behaviour

MobSF capture runtime logs, API calls, networks

v) Analyse Network Traffic

MobSF intercepts HTTPs traffic using its proxy.

6. Review Security findings

- Check for suspicious runtime behaviour
- Identify insecure API usage

7. Generate Reports :

Successfully performed dynamic analysis of the given mobile app using MobSF.

4) Results:

Obtained a detailed runtime security report consisting of- Network traffic analysis, runtime permission and API calls.

RS

Practical - 5

1) Title: Perform Reverse engineering with using APK tool, Hex Dump, and Dex Dump.

2) Aim:

To perform reverse engineering of a mobile application using the Reverse Engineering tools in order to analyse the internal structure and bytecode of the APK file.

3) Procedure:

1) APK tool Analysis

Install APK tool in the system ⇒

apktool d sample.apk

Install decompile the APK into manifest, resource and Smali code.

2) Hex Dump.

Use a hex editors or command line tool to generate a hex dump of APK file.

xxd sample.apk > sample_hex.txt

hook for signature, magic numbers or hidden data.

Examine the binary structure, headers and embedded strings

3) Dex Dump Analysis

- Extract the classes.dex from APK
- Use dexdump to analyse the dalvik bytecode

dexdump classes.dex > output.txt

- Review the output for class definition method and instruction

4) Documentation of findings.

Summarize permission, resource and bytecode analysis.

4) Results:

Operated all the following tools and analyzed the app and obtained the details like

- Application manifest and resource
- Binary structure and embedded data
- Dalvik bytecode and methodology.

RUN

Practical - 6

1) Title :- To perform security auditing using Drozer.

2) Aim :
To perform security auditing of android app using Drozer and identify vulnerabilities such as insecure components, exposed data and misconfigurations.

3) Procedure :

i) Step 1 : Install and setup Drozer.

- Install drozer on Kali
- Forward port using ADB

  adb forward tcp:31415 tcp:31415

- Connect drozer

  drozer console connect

ii) Step 2 : Identify Attack surface.

- List installed package.

  → run app.package.list

- Get package detail

  → run app.package.info -a com.eg.app

- Identify Attack surface

  → run app.package.attacksurface com.eg.app

iii) Step 3 : Enumerate Components.

- Activities

→ run app.activity.info -a com.eg.app

- Content Providers.

→ run app.providers.info -a com.eg.app

- Services

→ run app.service.info -a com.eg.app.

- Broadcast Receivers

→ run app.broadcast.info -a com.eg.app.

iv) Step 4 : Exploit Vulnerabilities

- Query Content Providers

→ run app.provider.query content://com.eg.provider/data

- Start exported activity

→ run app.activity.start --component com.eg.app
com.eg.app Activity Name

- Check for SQL Injection

→ run scanner.provider.injection -a com.eg.app.

4) Result:

Drozer was connected to the android to perform various components like activities services, broadcast receivers and content providers

Detected Security Vulnerabilities like :-

1. Exported components without permission
2. Sensitive data exposure via content providers
3. Provide injection flaws

R01

Practical - 7

Title : Perform Android application vulnerability assessment using QARK

Aim :-

To perform vulnerability assessment of android application using QARK and identify security flaws

Procedure :-

Step1 : Install QARK, clone from github

Step2 : Navigate directory and download requirement and run QARK

Step3 : Load Target App.

- Select option to scan an apk
- Provide path of the apk file
- Choose scan type

Step4 : Perform Vulnerability Analysis

QARK automatically analyses :

Android Manifest .xml

Application components (activities, services, receivers)

Permission and intent filter

Hardcoded sensitive data

Webviews vulnerabilities

Step5 :- Review Generated Reports.

After scanning, QARK generate a report (HTML format)

- Open the report and analyse.
  - Security level
  - Vulnerability description
  - Suggested fixes

4) Results :-

- Vulnerability assessment using SARK is performed and following vulnerabilities.
- Exported components without proper protection.
- Insecure permission usage
- Hardcoded sensitive information
- Weak cryptographic practices

Generated a detailed report with remediation suggestion

Practical - 8

1) Title : Analyze Network Traffic for android devices using HTTP toolkit

2) Aim:

To analyse and intercept network traffic of an android app using HTTP Toolkit and identify potential vulnerabilities.

3) Procedure:

i) Step 1 : Install and Launch HTTP toolkit

ii) Step 2: Configure device proxy and certificate

- Configure proxy settings on android.
- Install HTTPS certificate provided by HTTP toolkit
- Enable user installed certificate

iii) Step 3 : Capture Network traffic

- Start interception in the toolkit
- Open target android app
- Perform action (login, API calls)
- Observe captured traffic
  - HTTP/HTTPS requests
  - Headers and cookies

iv) Step 4 : Analyze traffic and Identify vulnerabilities

- Check for

Plaintext data transmission
Sensitive data
Insecure API data

2. Modify and replay request to test vulnerabilities

Results:

Observed API requests, headers & responses.

Identified potential vulnerabilities such as:

- Exposure of sensitive data
- Weak API security

RVT